

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv('/content/new_test.csv')
```

```
df.head()
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	duration	campaign	poutcome
0	32	4	0	6	0	0	0	0	3	3	131	5	1
1	37	10	3	6	0	0	0	0	4	3	100	1	1
2	55	5	0	5	1	2	0	0	3	2	131	2	1
3	44	2	1	0	1	0	0	1	4	3	48	2	1
4	28	0	2	3	0	0	0	0	5	0	144	2	1

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8238 entries, 0 to 8237
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   age             8238 non-null   int64
1   job             8238 non-null   int64
2   marital         8238 non-null   int64
3   education       8238 non-null   int64
4   default         8238 non-null   int64
5   housing         8238 non-null   int64
6   loan            8238 non-null   int64
7   contact         8238 non-null   int64
8   month           8238 non-null   int64
9   day_of_week     8238 non-null   int64
10  duration        8238 non-null   int64
11  campaign        8238 non-null   int64
12  poutcome        8238 non-null   int64
dtypes: int64(13)
memory usage: 836.8 KB
```

```
X = df.drop(columns=['poutcome']).values # input variable
y = df['poutcome'].values                # target variable
```

```
# Convert labels to {-1, +1} for SVM hinge loss
y = np.where(y == 0, -1, 1)

# Feature normalization (important for SVM convergence)
X = (X - np.mean(X, axis=0)) / np.std(X, axis=0)
```

```
# Set seed for reproducibility
np.random.seed(42)

# Shuffle indices
indices = np.random.permutation(len(X))

# 80-20 split
split = int(0.8 * len(X))
train_idx, test_idx = indices[:split], indices[split:]

X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]
```

```
def linear_kernel(x1, x2): #Linear → straight decision boundary
    return np.dot(x1, x2)

def polynomial_kernel(x1, x2, degree=3): #Polynomial → curved boundary
```

```

        return (1 + np.dot(x1, x2)) ** degree

def rbf_kernel(x1, x2, gamma=0.1): #RBF → complex non-linear separation
    return np.exp(-gamma * np.linalg.norm(x1 - x2) ** 2)

def sigmoid_kernel(x1, x2):# Sigmoid → neural-network-like behavior
    return np.tanh(np.dot(x1, x2))

```

```

class SVM:
    def __init__(self, C=1.0, kernel=linear_kernel, degree=3, gamma=0.1, lr=0.001, epochs=500):
        self.C = C
        self.kernel = kernel
        self.degree = degree
        self.gamma = gamma
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):
        self.X = X
        self.y = y
        self.alpha = np.zeros(len(X))
        self.b = 0

        # Kernel matrix
        self.K = np.zeros((len(X), len(X)))
        for i in range(len(X)):
            for j in range(len(X)):
                if self.kernel == polynomial_kernel:
                    self.K[i, j] = polynomial_kernel(X[i], X[j], self.degree)
                elif self.kernel == rbf_kernel:
                    self.K[i, j] = rbf_kernel(X[i], X[j], self.gamma)
                else:
                    self.K[i, j] = self.kernel(X[i], X[j])

        # Training using hinge loss
        for _ in range(self.epochs):
            for i in range(len(X)):
                decision = np.sum(self.alpha * y * self.K[:, i]) + self.b
                if y[i] * decision < 1:
                    self.alpha[i] += self.lr
                    self.b += self.lr * y[i]

    def predict(self, X):
        y_pred = []
        for x in X:
            decision = 0
            for i in range(len(self.X)):
                if self.kernel == polynomial_kernel:
                    decision += self.alpha[i] * self.y[i] * polynomial_kernel(self.X[i], x, self.degree)
                elif self.kernel == rbf_kernel:
                    decision += self.alpha[i] * self.y[i] * rbf_kernel(self.X[i], x, self.gamma)
                else:
                    decision += self.alpha[i] * self.y[i] * self.kernel(self.X[i], x)
            y_pred.append(np.sign(decision + self.b))
        return np.array(y_pred)

```

```

def metrics(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == -1) & (y_pred == -1))
    FP = np.sum((y_true == -1) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == -1))

    accuracy = (TP + TN) / len(y_true)
    precision = TP / (TP + FP + 1e-9)
    recall = TP / (TP + FN + 1e-9)
    f1 = 2 * precision * recall / (precision + recall + 1e-9)

    return accuracy, precision, recall, f1, [[TP, FP], [FN, TN]]

```

```

C_values = [0.01, 0.1, 1, 10, 100]

```

```

for C in C_values:

```

```

model = SVM(C=C, kernel=linear_kernel)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

acc, prec, rec, f1, cm = metrics(y_test, y_pred)
print(f"C={C} | Accuracy={acc:.3f} | F1={f1:.3f}")

```

```

C=0.01 | Accuracy=0.902 | F1=0.948
C=0.1 | Accuracy=0.902 | F1=0.948
C=1 | Accuracy=0.902 | F1=0.948
C=10 | Accuracy=0.902 | F1=0.948
C=100 | Accuracy=0.902 | F1=0.948

```

```

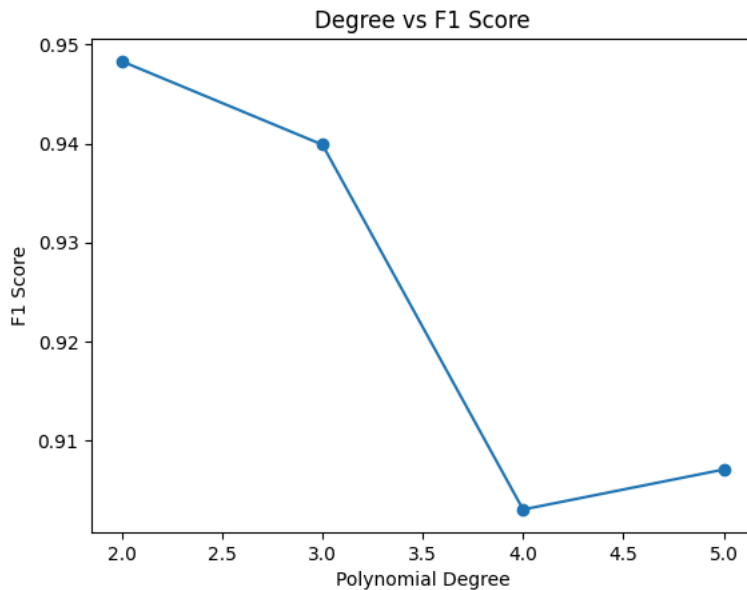
#Polynomial Kernel (Degree vs F1)
degrees = [2, 3, 4, 5]
f1_scores = []

for d in degrees:
    model = SVM(kernel=polynomial_kernel, degree=d)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    _, _, _, f1, _ = metrics(y_test, y_pred)
    f1_scores.append(f1)

plt.plot(degrees, f1_scores, marker='o')
plt.xlabel("Polynomial Degree")
plt.ylabel("F1 Score")
plt.title("Degree vs F1 Score")
plt.show()

```



```

C_vals = [0.1, 1, 10, 100]
gamma_vals = [0.001, 0.01, 0.1, 1]

results = []

for C in C_vals:
    for g in gamma_vals:
        model = SVM(C=C, kernel=rbf_kernel, gamma=g)
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        _, _, _, f1, _ = metrics(y_test, y_pred)
        results.append((C, g, f1))

results_df = pd.DataFrame(results, columns=["C", "Gamma", "F1"])
print(results_df)

```

	C	Gamma	F1
0	0.1	0.001	0.948309
1	0.1	0.010	0.948309
2	0.1	0.100	0.948309
3	0.1	1.000	0.948309
4	1.0	0.001	0.948309
5	1.0	0.010	0.948309
6	1.0	0.100	0.948309
7	1.0	1.000	0.948309
8	10.0	0.001	0.948309
9	10.0	0.010	0.948309
10	10.0	0.100	0.948309
11	10.0	1.000	0.948309
12	100.0	0.001	0.948309
13	100.0	0.010	0.948309
14	100.0	0.100	0.948309
15	100.0	1.000	0.948309

Start coding or [generate](#) with AI.